



Universidad
Tecnológica
del Perú

Ingeniería de Sistemas e Informática

Arquitectura Orientada al Servicio

**“Diseño e Implementación de una Plataforma Digital
Multiplataforma para la Gestión Integral de Operaciones en
Nachopps Restobar – Sullana, 2026”**

Autor(es):

Palacios Ynga, Marcos Roberto

Otero Reto, Juan Pablo

Requena Peña, Kevin Alexander

Docente:

Saavedra Navarro, Edwin Arnulfo

Piura– Perú, 2026

Índice

1. Resumen del caso

Nachopps Restobar es un negocio gastronómico ubicado en Sullana, Piura, que brinda servicio de restaurante y bar. A pesar de contar con clientela establecida y potencial de crecimiento, el negocio opera con procesos manuales que generan cuellos de botella operativos, errores frecuentes y pérdida de información crítica para la toma de decisiones.

La solución propuesta consiste en construir una plataforma digital orientada a servicios que digitalice y automatice las operaciones críticas: reservas, toma de pedidos, control de cuentas por mesa, gestión de caja, control de inventario y generación de reportes. De esta manera, se tendría una solución tecnológica integral basada en principios modernos de arquitectura de software, capaz de cubrir estas necesidades críticas de forma robusta, escalable y fácil de mantener.

2. Problema de Negocio y Objetivo

2.1. Problema Principal

Nachopps Restobar carece de un sistema integrado para gestionar sus operaciones. Los procesos actuales son manuales, lo que produce una cadena de ineficiencias que afectan la experiencia del cliente, la rentabilidad del negocio y la capacidad de escalar. Desde una perspectiva SOA, el problema central es la ausencia de servicios de negocio bien definidos: cada área opera como un silo aislado sin contratos ni interfaces de integración.

Síntomas observados:

- Reservas gestionadas de forma verbal o en papel, generando conflictos de disponibilidad.
- Pedidos tomados manualmente, con errores de transcripción y demoras en cocina/bar.
- Cuentas por mesa calculadas a mano, sin trazabilidad ni registro histórico confiable.
- Movimientos de caja no sistematizados: arqueos tardíos y errores en el cierre diario.
- Inventario sin control en tiempo real, con compras reactivas y pérdidas no monitoreadas.
- Ausencia de reportes para apoyar decisiones gerenciales.

Causas raíz identificadas

- No existe una arquitectura de servicios que integre los dominios operativos del negocio.
- Los flujos de información entre cocina, barra, sala y administración carecen de contratos formales.
- No hay visibilidad del estado del negocio en tiempo real ni mecanismos de integración entre áreas.

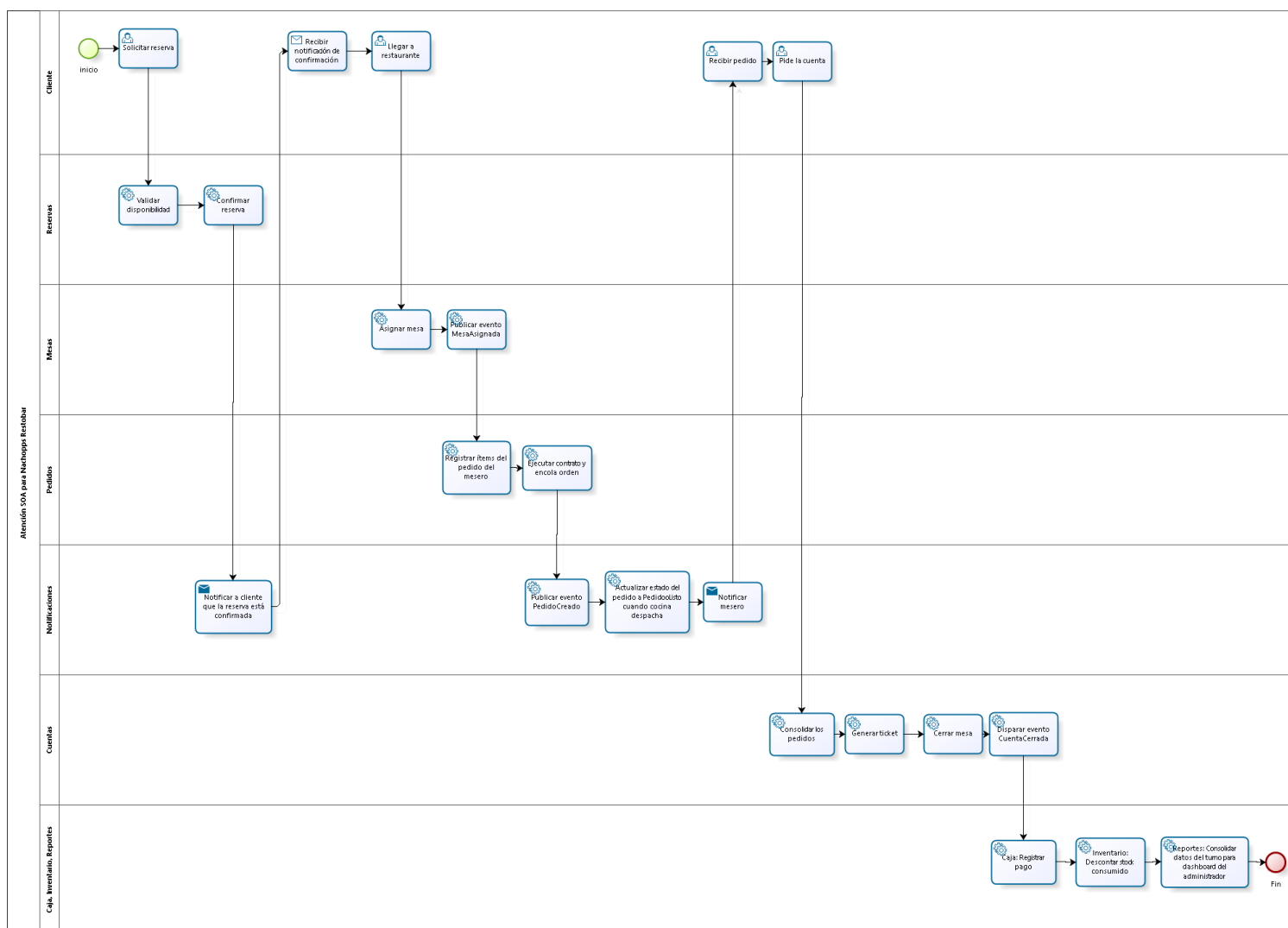
2.2. Objetivo General

Diseñar una solución orientada a servicios que integre las operaciones críticas de Nachopps Restobar mediante servicios de negocio autónomos, con contratos bien definidos, comunicación por mensajería y capacidad de operar de forma distribuida y escalable.

2.3. Objetivos específicos

- Identificar y delimitar los servicios de negocio que modelan las capacidades del restobar.
- Definir los contratos iniciales de los servicios principales.
- Establecer el mecanismo de integración entre servicios.
- Garantizar que cada servicio sea autónomo, reutilizable e interoperable.
- Diseñar la solución para soportar expansión a futuras sucursales sin rediseño estructural.

3. Mapa de capacidades y procesos clave



4. Servicios candidatos

En la arquitectura orientada a servicios, los servicios pueden ser clasificados según su finalidad, existen 3:

- Servicios de entidad: Son aquellos que encapsulan una entidad del negocio y su ciclo de vida
- Servicios de tarea: Orquestan un proceso de negocio específico invocando otros servicios.
- Servicios de utilidad: Proveen capacidades transversales reutilizables por todos los demás.

Teniendo esto en cuenta, se plantean los siguientes servicios:

Servicio candidato	Tipo de servicio	Capacidad que encapsula	¿Quién consume el servicio?
Reservas	Entidad	Ciclo de vida de reservas: creación, confirmación, cancelación, consulta de disponibilidad.	Cliente, Recepcionista
Mesas	Entidad	Estado de mesas (libre/ocupada/reservada) y asignación a cuentas activas.	Mesero, servicio de Reservas
Pedidos	Entidad	Registro de ítems por mesa, enrutamiento a cocina/bar y seguimiento de estado de preparación.	Mesero, Cocina, Bar
Cuentas	Entidad	Asociación de pedidos a mesa, división de cuenta, generación de ticket y cierre.	Mesero, Cliente
Caja	Tarea	Orquesta el flujo de pago: registra ingresos/egresos, gestiona arqueos y cierre de turno.	Cajero, Administrador
Inventario	Entidad	Control de stock: entradas, descuentos automáticos por pedido y alertas de nivel crítico.	Administrador, servicio de Pedidos
Reportes	Tarea	Consolida datos de Caja, Pedidos e Inventario para	Administrador, Gerencia

		generar dashboards y exportar reportes.	
Notificaciones	Utilidad	Entrega de notificaciones en tiempo real a los actores del sistema mediante mensajería.	Todos los servicios
Identidad	Utilidad	Autenticación de usuarios, autorización por rol (RBAC) y auditoría de acciones.	Todos los servicios

5. Límites iniciales de servicio

Servicio	Datos bajo su custodia	Operaciones expuestas	Mensajes que publica / consume
Reservas	Reserva {id, cliente, fecha, hora, mesa, estado}	crearReserva(), confirmarReserva(), cancelarReserva(), consultarDisponibilidad()	Publica: ReservaCreada, ReservaCancelada
Mesas	Mesa {id, número, capacidad, estado, cuentaAsociada}	consultarEstado(), asignarMesa(), liberarMesa()	Publica: MesaAsignada, MesaLiberada Consume: ReservaCreada
Pedidos	Pedido {id, mesa, ítems[], estado, timestamp}	crearPedido(), agregarÍtem(), actualizarEstado(), consultarPedido()	Publica: PedidoCreado, PedidoListo Consume: MesaAsignada
Cuentas	Cuenta {id, mesa, pedidos[], total, estado, ticket}	abrirCuenta(), agregarPedido(), dividirCuenta(), cerrarCuenta(), generarTicket()	Publica: CuentaCerrada, TicketGenerado Consume: PedidoListo
Caja	Movimiento {id, tipo, monto, medio, cuentaRef, timestamp}	registrarPago(), registrarEgreso(),	Publica: PagoRegistrado, ArqueoRealizado

		realizarArqueo(), generarReporteTurno()	Consume: CuentaCerrada
Inventario	Producto {id, nombre, stockActual, stockMínimo, unidad}	registrarEntrada(), consultarStock(), actualizarStock()	Publica: StockBajo Consume: CuentaCerrada
Notificaciones	Mensaje {id, destinatario, canal, contenido, estado}	publicarEvento(), suscribirseAEvento()	Broker central: Consume y enruta todos los eventos del sistema
Identidad	Usuario {id, nombre, rol, credenciales}, Token {jwt, expiry}	autenticar(), autorizarAcción(), registrarAuditoría()	Publica: UsuarioAutenticado
Reportes	Snapshot agregado de operaciones (no datos transaccionales)	generarReporteVentas(), generarReporteStock(), generarDashboard()	Consume: PagoRegistrado, CuentaCerrada, StockDescontado

6. Justificación del enfoque arquitectónico

Porque es el enfoque que mejor responde a la naturaleza del problema de Nachopps Restobar. El negocio opera con múltiples dominios funcionales que tienen ciclos de vida, actores y ritmos de cambio distintos, pero que necesitan integrarse para ejecutar procesos de negocio completos. SOA permite modelar esta realidad con precisión: cada dominio es un servicio autónomo con contrato propio, y la integración ocurre a nivel de contrato, no de implementación.

Necesidad del negocio	Principio aplicado	¿Cómo lo resuelve?
Dominios operativos con ciclos de vida distintos (cocina, caja, sala)	Autonomía del servicio	Cada servicio tiene su propia lógica, datos y despliegue. La caída de uno no bloquea a los demás.
Integración entre áreas sin acoplamiento fuerte	Contrato como única interfaz	Los servicios se comunican únicamente a través de contratos definidos (REST/OpenAPI + mensajería), no por dependencias internas.
Reutilización de capacidades transversales (autenticación, notificaciones)	Reutilización de servicios	Identidad y Notificaciones son servicios de utilidad consumidos por todos sin duplicar lógica.

Flujos de atención complejos que involucran múltiples servicios	Orquestación de servicios	Un proceso orquestador, como podría ser el flujo de cierre de cuenta, invoca servicios en secuencia según lógica de negocio.
Actualizaciones en tiempo real entre actores (cocina, mesero, barra)	Coreografía basada en eventos	Los servicios publican y consumen eventos de forma asíncrona sin necesidad de coordinación central.
Escalabilidad a futuras sucursales	Interoperabilidad y composición	Los servicios exponen contratos estándar que pueden ser consumidos desde otras sedes sin reimplementación.

7. Principios SOA Aplicados

Principio SOA	Descripción	Aplicación
Contrato estandarizado	Los servicios exponen interfaces formales independientes de su implementación.	Cada servicio define sus operaciones, entradas y salidas mediante OpenAPI/REST.
Bajo acoplamiento	Los servicios minimizan las dependencias entre sí.	La integración ocurre solo a través de contratos y eventos, nunca acceso directo a datos de otro servicio.
Abstracción	El consumidor no conoce la lógica interna del servicio.	El servicio de Pedidos no sabe cómo Inventario gestiona el stock; solo publica un evento.
Reutilización	Los servicios son diseñados para ser consumidos por múltiples consumidores.	Identidad y Notificaciones son servicios transversales reutilizados por todos los demás.
Autonomía	Cada servicio controla su lógica y datos sin depender de otros en tiempo de ejecución.	Caja puede procesar pagos, aunque Inventario esté temporalmente no disponible.
Stateless	Los servicios no retienen estado entre invocaciones.	Cada solicitud al servicio de Reservas es independiente; el estado se persiste en su propia base de datos.
Descubrimiento	Los servicios pueden ser localizados en tiempo de ejecución.	API Gateway actúa como registro de servicios y punto único de entrada para los consumidores.
Composición	Los servicios se pueden combinar para construir procesos de negocio complejos.	El flujo de atención completo se compone de 7 servicios orquestados/coreografiados.

8. Riesgos

Riesgo	Impacto	Probabilidad	Mitigación desde SOA
Definición incorrecta de límites de servicio (muy amplios o finos)	Alto	Alta	Usar Domain-Driven Design para validar cohesión.
Contratos mal definidos que generen acoplamiento implícito entre servicios	Alto	Media	Formalizar contratos OpenAPI y aplicar versionado de contratos desde el inicio.
Inconsistencia de datos en flujos distribuidos (ej. pago registrado pero stock no descontado)	Alto	Media	Aplicar patrón Saga con compensaciones transaccionales en flujos críticos orquestados.
Fallo del broker de mensajes (Kafka/RabbitMQ) afectando coreografía	Alto	Baja	Colas persistentes, reintentos automáticos y dead-letter queues para mensajes no procesados.
Conectividad limitada en el local	Medio	Media	Servicios de negocio críticos operan localmente con sincronización diferida a la nube al reconectarse.
Resistencia del personal al uso del sistema	Medio	Alta	Interfaces simples por rol; capacitación progresiva por módulo (servicio por servicio).